HYDRA ↔ SQUAD

Comparativa de los dos repositorios y hoja de ruta para trabajar sobre un fork de SQUAD

Iván Huarte · Universidad de Zaragoza · 11 de noviembre de 2026

> **Versiones comparadas. Hydra** = repositorio `NNs`, rama `dev` (commit `3787451` más los cambios locales: `PCvT`, `autoplots/aplot.py`, `index.py`, `reeval.py`); paquetes `Hydra/`, `CouplingModel/`, `Irreps/`, `autoplots/` (≈17,5 k líneas) y ≈130 tests en `pytests/`; manual `docs/01–11`. **SQUAD** = `../SQUAD`, rama `develop` (commit `bb4e2fd`, posterior a la release v0.4.0 del 9-nov), paquete `nqs` (≈11,8 k líneas), ≈13,4 k líneas de tests (≈1130 tests), 74 registros de decisión (ADR), manual de usuario, guía de desarrollador, arquitectura y CHANGELOG.

Contenido

1. Resumen ejecutivo
2. Filosofía y arquitectura general
3. Componentes modulares (directamente implementables)
4. Funcionalidades (lógica interna distinta)
5. Diferencias de convención que cambian los números
6. Balance en ambos sentidos
7. Hoja de ruta propuesta para el fork
8. Anexo: correspondencia de configuración Hydra → TOML de SQUAD

Leyenda de portabilidad (columna «→ SQUAD»: qué cuesta llevar el elemento de Hydra a SQUAD): `D` directo: se copia el código con cambios cosméticos (registro en la tabla de nombres, campos declarados, docstring, test); `A` adaptación: la lógica sirve, pero hay que reescribir interfaces o contratos; `R` rediseño: choca con la arquitectura o la filosofía de SQUAD y requiere una decisión (ADR) acordada con Alpesh; `✓` ya existe en SQUAD; `-` no recomendado / no aplica.

1. Resumen ejecutivo

- **Misma pila, distinto «pegamento».** Ambos usan NetKet 3.22 + JAX + Flax + Optax y el driver `VMC_SR`. Los componentes físicos (Hamiltonianos, reglas de Metropolis, capas Flax) son reutilizables casi tal cual. Las diferencias de verdad están en cómo se configura una simulación, cómo se orquesta el entrenamiento y qué se guarda.
- **Hydra es un laboratorio de investigación**: ≈30 módulos de red (transformers ViT2D, CvT3, PCvT…), proyección exacta de momento con *coset*, paridad Z2, 5 modelos de espín ½, callbacks de diagnóstico, precisión mixta, multi-GPU, barridos, herramientas de clúster y postprocesado (`aplot`).
- **SQUAD es un framework de ingeniería**: pocos componentes (2 backbones, 2 cabezas, 3 Hamiltonianos), pero cada uno con contrato, test y validación física. Su infraestructura está muy por encima de la de Hydra: un TOML validado entero antes de crear nada, registros inmutables con checksums, reanudación exacta, inferencia separada (`nqs-infer`), CI y ADRs.
- **SQUAD parte de una foto antigua de Hydra.** Su referencia de paridad es Hydra `b82edde` (estructura `NN_module/Factories/SingleModels`). Su triaje (ADR 0028) clasifica cada funcionalidad de Hydra como DONE / ON REQUEST / PARK / DROP. Gran parte de lo que Hydra ha ganado después no está contemplado: coset, `phase_mode` log/site, PCvT, opciones de CvT3, keeper con recuperación, precisión mixta, convolución depthwise, aplot y reeval.
- **Cinco bloqueos estructurales** impiden portar directamente las funcionalidades de Hydra: (1) el contrato de backbone `(B,N,C)` *por sitio* y con stride 1, incompatible con tokens y strides (ViT2D, CvT3, PCvT); (2) SQUAD no tiene una API de callbacks u observadores; (3) el driver está fijo (SGD + `VMC_SR` por defecto) y hay un único learning rate por periodo; (4) float64 es obligatorio; (5) un TOML corresponde a una ejecución sobre un único sistema, sin barridos.
- **SQUAD v0.5.0 (ADR 0072, decidido el 10-nov) va a cubrir parte de lo que te falta**: grupos de simetría (traslaciones, grupo puntual, inversión de espín), un wrapper de proyección,

composición de ansätze (secuencia, paralelo, residual, producto; cubre `Sequential`, `Transversal` y `SplitTraining`) y el registro de módulos Flax propios. **Coordínate con Alpesh antes de tocar Z2 o la composición**, para no duplicar trabajo.

- **Recomendación.** Haz el fork desde `develop` y trabaja «a la manera SQUAD» (una rama/PR por ítem, con su test y su documentación) para poder devolver cambios aguas arriba. Empieza por lo directo y de bajo riesgo: Hamiltonianos, samplers, schedule sigmoide y un lector de directorios de SQUAD para `aplot`. Después ataca el contrato de «malla gruesa + coset», que es lo que desbloquea tus transformers (sección 7).

2. Filosofía y arquitectura general

Aspecto	Hydra (NNs)	SQUAD (nqs)
Objetivo de diseño	Flexibilidad para investigar: «todo elemento nuevo debe poder controlarse desde los cuatro JSON».	Reproducibilidad y auditabilidad: «el fichero es datos»; todo problema se detecta antes de crear nada; ningún cambio numérico silencioso (regla G5).
Configuración	4 JSON en orden fijo (config0 simulación, config1 modelo, config2 red/stages, config3 storage/ simetrías). Patrón `selection`: los hermanos no seleccionados son un catálogo inactivo. Learning rates como cadenas (`"exp(0.007,0.0005)"`).	1 TOML con `schema_version`. Cada tabla con `name` elige un componente de una tabla de nombres; las claves permitidas son exactamente los campos declarados del componente (una errata es un error). Los schedules son tablas de datos, nunca se evalúan cadenas.
Punto de entrada	`VMC_simulation.py -c ×4`	`nqs-run file [--dry-run]`, `nqs-run --resume dir`, `--resume-from ckpt`, `nqs-infer file`. Las mismas funciones están disponibles como API de Python.
Validación	En tiempo de ejecución: un código erróneo de un stage posterior falla al llegar a ese stage.	Cinco niveles antes de crear nada: parseo, validación, construcción, consistencia física y construcción en seco (inicializa cada red y prueba cada transferencia).
Estructura de paquetes	`Hydra`, `CouplingModel`, `Irreps` y `autoplots` se importan desde la raíz y no se instalan.	Un paquete instalable, con niveles 0–6 de dependencias (`errors → runtime/systems → symmetry/statistics → ansatz/reference → sampling/vmc → training/inference → experiment`). Un test prohíbe los imports hacia arriba.
Extensión	Diccionarios `REGISTRY_*` más listas `LATTICE_SIZE/HILBERT`. Hay subcadenas reservadas en los nombres (`Seq`, `Phase`, `NN`…).	Una tabla de nombres por paquete (`HAMILTONIANS`, `SAMPLERS`, `SCHEDULES`, `ACTIVATIONS`, `OBSERVABLES`, registro de ansatz), con contrato documentado y un test de registro y de forma.
Estado global	`DTYPE` global por módulo y `precision._MODULES`; float64 activado en el script.	Importar no cambia nada; `param_dtype` es un campo de cada módulo; se rechaza ejecutar con float64 desactivado.
Errores y logs	`print/rich`, `assert`, `KeyError`.	Excepciones tipadas (`ConfigurationError`, `ContractError`, `NumericalError`, `ResourceError`, `RunStorageError`…). Log con ubicación y contexto `[stage period step]` en `run.log` y `run.jsonl` (eventos). Perfiles user/ developer/trace. Un fallo deja `failure.json`.
Tests	≈130 tests por áreas (equivariancia, keeper, multistage, irreps, schedule…).	≈1130 tests: unitarios, de integración (VMC corto), de conformidad física, oráculos de paridad bit a bit con Hydra `b82edde` y de tooling. Además, benchmarks contra ED con el criterio fijado antes de ejecutarlos.

Aspecto	Hydra (NNs)	SQUAD (nqs)
Proceso	Un desarrollador; ramas `dev`/`main`.	PR con commits firmados; CI en GitHub Actions (ruff, tests, wheel, docs); pre-commit; ADRs; SOP; CHANGELOG; releases etiquetadas; licencia Apache-2.0; CITATION.
Documentación	Manual de 11 capítulos (castellano), práctico y orientado al uso.	Manual de usuario, guía de desarrollador, glosario, arquitectura, derivación matemática (PDF), notas por componente e informes de benchmark.

3. Componentes modulares (directamente implementables)

Son piezas que, por su naturaleza, encajan en una tabla de nombres de SQUAD: basta escribir una función o un módulo, registrarlo y añadir su test. Columna «→ SQUAD»: coste de llevar lo de Hydra a SQUAD.

3.1 Redes cristalinas y espacios de Hilbert

Elemento	Hydra	SQUAD	→ SQUAD	Nota
Cuadrada, cadena, triangular	`Square`, `Chain`, `Triangular`; contorno abierto o periódico	`square`, `chain`, `triangular`	✓	La numeración row-major coincide con `default_2` de Hydra.
Honeycomb, kagome, cúbica	–	✓ (varios sitios por celda)	✓	Sólo en SQUAD.
Órdenes de sitio alternativos	`default_1/2`, `snake_1/2`	Una numeración fija, con un único módulo dueño	-	Innecesario si se usa row-major.
Vecinos NN, NN2, NN3	Listas de desplazamientos por orden	`Lattice.netket_graph(max_neighbor_order)`	✓	SQUAD colorea los enlaces por orden de vecino, no por dirección (relevante para Oxalate).
Espín ½ con S^z total fijo	`sz_total`	`total_sz`	✓	
Espín S arbitrario, bosones	–	`spin` (S), `boson` (`n_max`, `n_particles`)	✓	Sólo en SQUAD.

3.2 Hamiltonianos

Modelo	Hydra (`CouplingModel`)	SQUAD (`systems/hamiltonians`)	→ SQUAD	Nota
Ising transverso	`IsingSquare`: $\Sigma\, h\, \sigma^{op} + J\, \Sigma\, \sigma^z\sigma^z$, operadores configurables (`ops`)	`transverse_field_ising`: $-h\, \Sigma X + J\, \Sigma ZZ$	✓/D	La versión con operadores generales (campos en X/Y/Z) es directa.
Heisenberg J1-J2	`J1J2Square` con campos (h_x, h_y, h_z); unidades de Pauli por defecto (`S_operators`)	`heisenberg_j1j2` en cualquier red y cualquier S; operadores S; sin campos	A	Añadir campos como modelo nuevo o como campo opcional. **Unidades: $E_{Pauli} = 4 \cdot E_S$.**
Shastry–Sutherland	`SSMSquare` (J, Jp, campos)	–	D	Los dímeros se cierran periódicamente aunque el contorno sea abierto; hay que decidirlo explícitamente.
Largo alcance	`LRSquare`, `LRChain`: $J\, r^{-\alpha}/\mathcal{N}$ (Kac)	–	D	Una función que recorre todos los pares.
J-K-Γ (Oxalate)	`Oxalate`, triangular, términos según la dirección del enlace	–	A	Necesita la dirección de cada enlace; `Lattice` sólo da su orden.
Bose–Hubbard	–	`bose_hubbard`	✓	Sólo en SQUAD.
Representación	Arrays densos Q×Q de acoplos h^o_i, J^o_{ij}; `ModelFactory` genera los puntos del barrido	Funciones `model(lattice, hilbert, *, params)` que devuelven un operador NetKet nativo	A	Portar = reescribir cada modelo como función, más un test del espectro contra una matriz construida de forma independiente.

3.3 Redes neuronales

El **contrato de SQUAD** es `embedding` $(B,N)\to(B,N,C_{in})$ → `backbone` $(B,N,C_{in})\to(B,N,C)$ → `head` $(B,N,C)\to(B,)$, más una regla de signo opcional. El backbone debe conservar el eje de sitios: si cambia ese eje, se lanza `ContractError`. En Hydra la entrada es $(B,L0,L1,1)$ y un backbone puede devolver una malla reducida (B,H',W',C) o tokens (B,N',C). Esta es la diferencia clave.

Módulo	Hydra	SQUAD	→ SQUAD	Nota
MLP	`MLP` (anchura α·N, `final_architecture`, `only_phase`)	`mlp` local por sitio, `widths` absolutas	✓	Distinto árbol de parámetros (paridad probada salvo nombres).
CNN	`CNN` (strides, pooling)	`cnn` periódica, stride 1, sin pooling	✓	
CMLP	Perceptrón complejo	–	D	ON REQUEST en el triaje.
CNNLiang	Liang et al. 2021, con `window_pooling`	–	A	Directo con `window_pooling=1`; con pooling rompe el contrato por sitio.
ViT2D	Atención factorizada circulante; tokens t×t; salida (B,H/t,W/t,d)	–	A	Los tokens rompen el contrato (B,N,C): necesita el contrato de malla gruesa (§4.2).
CvT3 (y CvT, CvT2)	Embedding convolucional con stride, Q/K/V depthwise, opciones `ffn_out/norm/attn_out_proj`	– (ON REQUEST «un cvt»)	A	Igual que ViT2D. Se lleva consigo `toolbox.DepthwiseConv`, que esquiva el bug de JAX con convoluciones agrupadas bajo la malla de NetKet.
PCvT + SiteHead	Atención con sesgo posicional relativo; pesos independientes del tamaño de la red (6x6 → 8x8)	–	A	Posterior al triaje. Encaja bien con la transferencia por ruta de SQUAD entre tamaños (mismas formas).
CvTaps, CvTexp, VViT	Experimentales; VViT probablemente roto	– (PARK)	-	No portar.
Jastrow, Factorized, FactorMod	Ansätze simples sobre entrada plana	–	A	Son funciones de onda completas, no backbones: necesitan el «producto de funciones de onda» de v0.5.0.
Redes de fase (CNNPh, EDPPh, CNNClsf, CNNbinClsf, CNNSzabo)	Experimentales	– (PARK)	-	Baja prioridad.
ARViT (autorregresivo)	Con `ARNNWrapper` y muestreo directo	–	R	SQUAD sólo tiene muestreadores de Metropolis.
ComplexHead	Suma → LN → Dense complejo → Σ log cosh	`complex_head` (exige C ≥ 3)	✓	
OutputHead, DeepOutputHead	Ramas Re/Im Dense+LN → Σ log cosh (cabeza estándar)	–	D	Cumplen el contrato de cabeza tal cual (suman sobre tokens o sitios).
SzaboOutput, Sum, Mean, FinalFF	Reductores	–	D	

Módulo	Hydra	SQUAD	→ SQUAD	Nota
Proyección de momento	`Sequivariant`: `phase_mode` angle/log/site, `coset`	`momentum_phase` ($\psi = z \cdot \psi_{head}$; sólo red cuadrada)	A	SQUAD equivale a `phase_mode="log"` sin coset (§4.2).
Composición (flujo)	`Sequential`, `SingleModule`, `SplitTraining`, `Transversal` (sum, product, sum_angles, modulus_marshall…), `SequivariantX`	– (prevista en v0.5.0, fase 5)	R	Coordinar con Alpesh.
Envoltorio Z2	$\psi(\sigma) \pm \psi(-\sigma)$, `symm_modulus`	– (inversión de espín prevista en v0.5.0)	A	El módulo es trivial; lo costoso es la contabilidad de sectores, el ED con Z2 y los chequeos de consistencia.
Envoltorio Traslation, CarreteSign	No funcionales	–	-	
Signo de Marshall	Módulo `MarshallSign`	Regla de signo declarada y verificada (red bipartita, opción `approximation`, compatibilidad con el sector de momento)	✓	La versión de SQUAD es más segura.
Embedding one-hot	–	`identity`, `one_hot`	✓	Sólo en SQUAD.
Activaciones	swish, gelu, relu, elu, tanh, log_cosh, cardioid, ModReLU, activaciones complejas separadas	gelu, relu, swish, tanh	D	Añadir a `ACTIVATIONS`.

3.4 Muestreo: reglas de propuesta

Regla	Hydra	SQUAD	→ SQUAD	Nota
Volteo local	`LocalRule`	`metropolis_local`	✓	
Intercambio	`Exchange`: elige una capa de vecinos con pesos (≈64/29/6 %)	`metropolis_exchange` con `d_max`	D	Las capas ponderadas, como regla nueva.
Intercambio J1-J2	`ExchangeJ1J2`: NN con probabilidad J1/(J1+J2), NNN en otro caso	–	D	
Inversión global	`InvertMagnetization`	–	D	Útil mezclada con otras reglas.
Mezcla de reglas	`MultipleRules` con `prules`	–	A	Requiere un campo de configuración «lista de reglas con probabilidades».
LocalRule_Z2	Volteo salvo el sitio 0	–	D	Sólo tiene sentido con Z2.
Movimientos del Hamiltoniano, parallel tempering	–	`metropolis_hamiltonian`, `parallel_tempering_hamiltonian`	✓	Sólo en SQUAD; además comprueba la ergodicidad hasta 2^{18} estados.
Estado inicial de las cadenas	`hilbert`, `M0`, `Mmax`, `hilbert_even_M`	Aleatorio (NetKet)	A	Afecta a la reproducibilidad: debe quedar registrado.
Muestreo directo autorregresivo	`nkARSampler`	–	R	Va con ARViT.

3.5 Schedules y optimizador

Elemento	Hydra	SQUAD	→ SQUAD	Nota
Constante, lineal, exponencial	`lin(a,b[,S])`, `exp(a,b[,S])`, con escalera opcional	`{form="constant|linear|exponential"}`	✓	La escalera es un cambio pequeño.
Sigmoide	`sig(y0,y1,x_d)` (tu receta de diag_shift)	–	D	Nueva entrada en `SCHEDULES`.
diag_shift por eon o por periodo	Una curva global, una por eon o una por periodo	Siempre una por periodo	✓	Una curva global se parte en tramos.
Optimizador	`sgd`/`adam` más modificaciones de optax (clip, scale, trace…)	SGD fijo	A	Campo nuevo de `Period` (guía de desarrollador §8).
Opciones del SR	`linear_solver`, `chunk_size_bwd`, momentum SPRING, `use_ntk`, `on_the_fly`, `proj_reg`, `mode`; también `VMC + SR(cg)`	`VMC_SR` con los valores por defecto y sólo `diag_shift`	A	Imprescindible para CvT3 8x8 (memoria). Debe registrarse en `specification.json`.
Learning rate por rama	Un lr por código en cada periodo	Un lr por periodo (ON REQUEST)	A	

3.6 Observables y diagonalización exacta

Elemento	Hydra	SQUAD	→ SQUAD	Nota
m, m²	`M_vs`, `M2_vs` (Pauli)	`magnetisation`, `magnetisation_squared` (unidades S)	✓	Factor 2 de unidades.
m escalonada ms, ms²	Signo $(-1)^i$ (rayas en anchura par)	–	D	Mejor leerla de S(q) en el q adecuado.
Correlaciones de espín	`correlations_vstate`/`_ED`: $\langle\sigma_i\cdot\sigma_j\rangle$	`spin_correlation`, `spin_correlation_z`	✓	
Factor de estructura S(q)	Scripts antiguos (`calc_SSF_NN.py`, obsoleto)	`structure_factor`, `_z`	✓	Sólo en SQUAD (mantenido).
Entropía de Rényi-2	`renyi_vs` (sin S^z fijo)	–	A	No es un operador: necesita un tipo nuevo de observable (estimador de réplica).
Estadísticas de módulo y fase	`modphase`, `phase_stats`, histogramas ponderados	–	A	Como medida de `nqs-infer`.
Fidelidad	Global y de sector	Del sector, con el gap al siguiente nivel	✓	
Utilidades de análisis	`distance_with_neel`, `print_max_contributors`, `full_basis_state`	–	D	Herramientas fuera del paquete.
ED global	Lanczos	`lowest_states`	✓	
ED por sector de momento	Proyector sin matrices densas + Lanczos sobre HP + c(1−P)	Base de órbitas dispersa (red cuadrada, sólo traslaciones)	✓	Algoritmos distintos; coinciden a 1e-10.
ED con Z2	Sectores (T_x, T_y, Z2)	–	A	Extender la base de órbitas al grupo T×Z2.
Inspección del espectro	`ED_inspection.py` (10 niveles, figuras)	–	D	Script independiente.

Elemento	Hydra	SQUAD	→ SQUAD	Nota
Energía exacta durante el entrenamiento	–	`exact_energy_every` + aviso si el valor muestreado y el exacto difieren	✓	Sólo en SQUAD.
Conmutación de H con las traslaciones	–	Comprobación exacta, sin listar configuraciones	✓	Sólo en SQUAD.

4. Funcionalidades (lógica interna distinta)

Aquí el problema no es el código de un componente, sino cómo se orquesta. Para cada funcionalidad se describe qué hace cada repositorio y cómo llevarla a SQUAD sin romper su diseño.

4.1 Construcción dinámica de redes

Hydra	SQUAD
• La plantilla JSON es un árbol arbitrario de módulos de flujo y módulos simples; los hiperparámetros salen de `storage[clase]`. Admite sufijos (`MLP_2`) y referencias a otro stage (`{"stage0":{}}`). • Inyecta automáticamente `lattice_size` y `hilbert`. `NNWrapper` reordena `(B,N)→(B,L0,L1,1)`. • Los envoltorios de simetría se apilan en el orden de config3. • `change_attr` (con `_count`) modifica atributos entre stages. • Códigos `code2path` (`"0"`, `"01"`, `"A"`) para entrenar o trasplantar subárboles. • Diagrama de cajas con los códigos, árbol de parámetros y recuento de parámetros.	• Composición fija: embedding → backbone → cabeza (+ regla de signo). Una cabeza puede envolver a otra (`momentum_phase.head`). • `NetKetWavefunction` es el único adaptador `(B,N)↔` red; la geometría la da `CellGrid`. • Rutas de parámetros con nombre (`wavefunction/backbone/ConvBlock_0/Conv_0/kernel`), fijadas por el módulo que las crea. • La construcción en seco valida forma y tipo de cada componente (`ContractError`); hay un recuento de parámetros por componente. • No hay árboles arbitrarios: la composición está en el alcance de v0.5.0.

> **Cómo llevarlo a SQUAD.** No portes el sistema de plantillas, *storage* y códigos: es justo lo que SQUAD sustituyó por diseño (DROP: «positional codes», «four-file configuration»). Las rutas de SQUAD hacen el papel de tus códigos y son más legibles y robustas ante reordenaciones (`trainable_prefixes`, `transfer_rules`). Lo que sí aporta valor es la **composición**: participa en el diseño de v0.5.0 para que cubra `Transversal` (sum/product/sum_angles) y `SplitTraining`. El diagrama de arquitectura encaja como ampliación del informe de `--dry-run`, que la propia arquitectura de SQUAD prevé como «dirección futura».

4.2 Simetrías y sectores (el punto más importante)

Hydra	SQUAD
• `Sequivariant`: el backbone devuelve una malla equivariante `(B,H,W,C)`; $z = \Sigma_r \chi_k(r)\cdot x[r,0]$ y la cabeza es invariante. • `phase_mode`: `angle` (log ψ = cabeza + i·arg z), `log` (ψ = $e^{\text{cabeza}}\cdot z$) o `site` (SiteHead, una amplitud por sitio). • **`coset`**: suma sobre los desplazamientos dentro del stride. Con backbones de stride s da un autoestado exacto de momento de la red completa; sin coset, k y k+2πa/s se mezclan. • Envoltorio Z2 (par/impar). El sector de cada stage se deduce de la plantilla (`stage_sectors`) para el ED. • Tests de equivariancia para CvT3, CvT, ViT2D y CNNLiang.	• `MomentumPhase(head, sector)`: $\psi_m = z\cdot\psi_{\text{head}}$, con z la componente de Fourier del canal 0 *por sitio*. En m = 0 devuelve la cabeza sin cambios. • Convención fijada por la física (ADR 0021: $\chi_m(a) = e^{-iq\cdot a}$, traslación activa). Hydra `b82edde` se comprobó contra ella. • Chequeos de consistencia: H conmuta con las traslaciones; la regla de signo no saca ψ de su sector. • Sólo red cuadrada; sin coset; backbone por sitio (stride 1). • v0.5.0: grupos (traslación, punto, inversión de espín), wrapper de proyección (previsiblemente `nk.nn.blocks.SymmExpSum`), informe de qué simetrías respeta cada componente y coste en evaluaciones en el `--dry-run`.

> **Bloqueo 1: contrato por sitio.** Tus mejores redes (CvT3 con strides [[2,2],[1,1]], ViT2D con tokens 2×2, PCvT) producen una malla *gruesa*. En SQUAD eso es un `ContractError`. **Propuesta para Alpesh** (ADR conjunto): (a) un contrato adicional de «backbone sobre malla», con salida `(B,`

n_{celdas}, C) sobre un CellGrid reducido y un stride declarado; (b) que el wrapper de proyección de v0.5.0 acepte un *subgrupo* de traslaciones. El coset de Hydra es exactamente la proyección sobre el subgrupo de desplazamientos dentro del stride, combinada con la fase de Fourier sobre la malla gruesa. Así el coset no necesita código especial, y MomentumPhase queda como el caso stride = 1. Los tests de equivariancia de Hydra se convierten en los tests de contrato de esos backbones.

4.3 Etapas y trasplante de pesos

Hydra	SQUAD
• Eon del schedule = stage de config2; arch_evol crea un MCState nuevo en cada cambio. • load: [origen, código_viejo, código_nuevo]. Por defecto se trasplanta todo lo compatible; [] = inicialización aleatoria. Con save_params se puede tomar de cualquier stage anterior y, en stage0, de fuentes externas (stageX<i>). • greedy_transplant: primero misma ruta relativa, después el primer hueco libre de forma compatible (puede emparejar arrays no relacionados). Convierte el dtype. Lo no rellenado se lista como «random init». • load_model acepta JSON de resultados, carpeta, glob, _best.orbax[/modo[/stage]] o un paso de _checkpoint.orbax, y también el estado del sampler. • Keeper *recover*: el stage siguiente parte del mejor estado si el final es inestable.	• Stages inmutables. transfer_parameters trabaja por ruta, con reglas source→target (renombrados), fresh_prefixes y discard_prefixes. • **Cada hoja debe quedar justificada**: copiada, declarada nueva o descartada. Una diferencia de forma o de dtype es un error, y se detecta antes de empezar. • Escribe transfer.json con el informe y registra el salto de energía ΔE ± error en cada frontera. • Siempre parte del θ *final* del stage anterior, nunca del mejor (el mínimo de estimaciones ruidosas está sesgado hacia abajo, ADR 0028 B9). • initial_parameters = {from=run, stage=k} sólo para el primer stage, con checksums verificados. Optimizador y cadenas empiezan de cero en cada stage. Sin crecimiento de anchura.

Cómo llevarlo a SQUAD. Su transferencia es más segura que greedy_transplant; no hace falta portar este último. Lo que falta son ampliaciones A: (i) varias fuentes externas hacia una misma red (tu stageX<i>); (ii) partir de un checkpoint concreto o de un mejor estado, y no sólo del final de un stage; (iii) conversión explícita de dtype; (iv) transferir el estado de las cadenas. El *recover-from-best* choca con B9 R: plantéalo como opción explícita y registrada, nunca por defecto. Tus arranques en caliente entre tamaños con PCvT funcionan ya con initial_parameters, porque las formas no dependen del tamaño de la red.

4.4 Entrenamiento, schedule y congelado

Hydra	SQUAD
• Listas anidadas [eon][era][periodo] de épocas, modos (códigos) y lr (uno por código); rescale. • Cada periodo crea un optimizador y un driver nuevos. • **Congelado por máscara**: el SR se resuelve con todos los parámetros y después se anulan los congelados (optax.set_to_zero). • survive_condition detiene el periodo si V < baseline o hay NaN, pero los periodos siguientes continúan.	• stages → periods (n_steps, learning_rate, diagonal_shift, trainable_prefixes). Los schedules son funciones de t, comprobadas antes del primer paso. • **SR restringido**: sólo se resuelve el bloque activo. Es el gradiente natural con los congelados fijos; da números distintos (un 40 % de diferencia en el paso en 3×3 con λ = 10^{-3}). • NaN → NumericalError, que detiene la ejecución; una aceptación < 1 % o una varianza nula generan aviso.

Cómo llevarlo a SQUAD. El schedule de Hydra se traduce 1:1 al de SQUAD salvo dos cosas: el lr por rama y una curva de diag_shift que cruza stages (se parte en tramos). Las opciones de SR y de optimizador necesitan campos nuevos en Period A; la guía de desarrollador (§8) ya indica dónde van y que obligan a subir la versión del estado de entrenamiento si el driver guarda estado (p. ej. el momentum SPRING). **Ojo:** resultados de Hydra con módulos congelados no son comparables con los de SQUAD.

4.5 Muestreo (lógica)

Hydra	SQUAD
• n_samples = nº GPUs × `n_chains_per_rank` × `n_samples_per_chain` (típico: 1024 cadenas × 1 muestra). • **`n_discard_per_chain = 0`**; barrido de N propuestas. • `chunk_sampler`, `chunk_vstate`; estado del sampler guardado y recargable; las cadenas se conservan entre periodos.	• `n_chains` (16 por defecto); `n_samples` múltiplo de `n_chains`; `sweep_size`. • **`n_discard_per_chain = 16`** (valor medido: 5 sesgaba); `chunk_size`. • Rechaza muestreadores que romperían la restricción o no serían ergódicos. • $\hat{R}$ y τ en cada paso; medida de τ_{int} por cadena al final de cada stage (muestras efectivas). • Cadenas nuevas en cada stage (ADR 0028 B3).

4.6 Precisión, memoria y hardware

Hydra	SQUAD
• `network_dtype: float32`: precisión mixta (parámetros, Jacobiano y SR en float64). Unas 4× más rápida en RTX 3080 y L40S. `matmul_precision: highest` para evitar TF32. • Multi-GPU (sharding de NetKet; los estados se restauran como numpy en cualquier número de GPUs). • `chunk_size_bwd`; recetas de memoria documentadas; `DepthwiseConv` para el bug de JAX.	• float64 obligatorio, con `param_dtype` por módulo. • `chunk_size`; ante un OOM lanza `ResourceError` sugiriendo un divisor. • Benchmark de rendimiento de referencia y coste de las métricas medido. • Reproducibilidad bit a bit sólo en CPU (documentado). • Multi-GPU no documentado ni probado (benchmarks en una GPU de portátil).

> **Bloqueo 4: float64.** Para garmonegro (L40S, float64 lento) la precisión mixta es esencial. En SQUAD cambia números, así que necesita un ADR: un campo `dtype` de cómputo separado de `param_dtype` en cada módulo, registrado en `manifest.numerics`, y un benchmark que demuestre que la trayectoria coincide con float64 (tú ya lo mediste en CvT3). Para multi-GPU basta con un test y documentación (divisibilidad de `n_samples` por dispositivo).

4.7 Callbacks y monitorización

Funcionalidad de Hydra	Qué hace	Equivalente en SQUAD	→ SQUAD
BestIterKeeper	Mejores estados por criterio (`best` E+|E|V/c, `best_energy`, `best_vscore`, `always`), por stage; sólo estados físicos; *recover*; escritura atómica; `_trace.txt`.	Ninguno, por diseño (se usa el θ final). La traza paso a paso está en `run.jsonl` (en vivo) y en `metrics.csv`.	R
Checkpoint	Orbax asíncrono, `max_to_keep`, `keep_every`, `stage_end`.	Checkpoints cada 50 pasos o 10 min y al final de cada stage; reanudación exacta (§4.8).	✓
SanityMonitor	Energía, muestreo, optimización por módulo, función de onda, gradientes y espectro del NTK, colas de E_{loc}, magnetización.	`StepMonitor` cubre energía y muestreo (NaN, aceptación, $\hat{R}$, τ, V, norma del paso SR). El resto, nada (SanityMonitor está en DROP).	R
LeavesGradient	Paso relativo por hoja y módulo, hojas atípicas, mapa de calor final.	Sólo la norma global $\|\delta\theta\|$ (`sr_update_norm`).	R
EnergyPlotter / ModPhasePlotter	Ventanas en vivo y PNG; módulo y fase frente al ED.	Ninguno («plotters inside the training loop»: DROP).	A
Gráfica final (`Callback_*.jpeg`)	Energía, error, V, schedule, mejor paso.	Ninguno.	A

Funcionalidad de Hydra	Qué hace	Equivalente en SQUAD	→ SQUAD
(sólo en SQUAD)	–	Aviso de energía muestreada frente a exacta; medida de τ_{int}; `performance.json`; `failure.json` con los últimos 200 eventos; perfil `trace` que nombra el primer parámetro no finito.	

> **Bloqueo 2: no hay API de observadores.** El manual de SQUAD (§9) lo dice: «no progress callback». La arquitectura (§6) prevé «observers called after each SR step» como dirección futura. Es la pieza que desbloquea el keeper, LeavesGradient y un SanityMonitor por módulo; propónla como ADR temprano. Mientras tanto, **las gráficas en vivo no necesitan tocar `nqs`**: un visor externo que lea `run.jsonl` (un evento `sr.step` por paso) encaja exactamente con la filosofía de SQUAD de «clientes separados que leen ficheros». El keeper puede reformularse como un observador que guarda instantáneas en el directorio de la ejecución, sin usarlas por defecto para transferir. La re-evaluación con muchas muestras (tu `reeval.py`) ya resuelve el sesgo que preocupa a SQUAD.

4.8 Checkpoints, reanudación y robustez

Hydra	SQUAD
• Checkpoints orbax; se pueden cargar como *arranque en caliente* de una simulación nueva, pero no hay reanudación exacta. • SIGTERM → `SystemExit`, que vuelca los mejores estados; el JSON de resultados no se escribe si la simulación cae, pero la carpeta sirve como `load_from`.	• `--resume`/`--resume-from`: en CPU se reproduce byte a byte. El estado guardado incluye optimizador, cadenas con su clave aleatoria y la historia del periodo. • Antes de reanudar comprueba el lock, el config, el entorno (commit git, `uv.lock`, versiones) y los checksums. • SIGTERM/SIGHUP → checkpoint y estado `interrupted` (salida 143); Ctrl-C → 130. • `run.lock`, `status_history`, `lineage`.

Para el clúster esto es una ventaja clara de SQUAD: un trabajo que agota su límite de tiempo en Slurm se reanuda exactamente, algo que en Hydra no es posible. Cuidado con el lock en sistemas de ficheros de red (issue #287, previsto en v0.5.0).

4.9 Resultados, almacenamiento e inferencia

Hydra	SQUAD
• Jerarquía automática `<raíz>/<Modelo>/<sel>_<arq>/Size_LxL/ST_…/UUID_…/`; las etiquetas codifican los parámetros y los scripts las parsean. • JSON de resultados (`CM`, `NN.setup`, `SIM`, `results`, `_artifacts`, `labels`, `metadata`); parámetros en orbax; copia de los 4 configs; vectores ED en `.txt`. • Medidas al final del entrenamiento sobre el mejor estado (`measureNdump`): observables, ED, módulo y fase, Rényi. `load_vstate` para scripts; `reeval.py` con muchas muestras.	• El usuario elige `run_directory` (debe estar vacío). • `manifest.json`: estado, semilla, config resuelto, commit y si había cambios sin commit, hash de `uv.lock`, versiones, numéricos, linaje y SHA-256 de cada fichero. • `stages/NNN/`: `specification`, `result`, `metrics.csv`, `parameters.msgpack`, `transfer.json` y `performance.json`, escritos de forma atómica (`.partial`) y con versión de esquema. • `nqs-infer`: TOML propio, muestras nuevas con semilla propia, todas las medidas sobre las mismas muestras, valores exactos y fidelidad ($\leq 2^{20}$ estados); carpeta propia con manifest. Rechaza una fuente que no coincida con lo registrado.

→ **SQUAD:** tu `reeval.py` equivale a `nqs-infer` sin observables. Rényi y módulo/fase serían tipos de medida nuevos **A**. La organización automática de carpetas por modelo, tamaño y parámetros no debe ir en `nqs`: la decide el generador de campañas al escribir `run_directory`.

4.10 Barridos y campañas en clúster

Hydra	SQUAD
• `sizes` y claves `*_list` con modo `for`/`zip`: muchos puntos por trabajo, un UUID por tamaño. • `Simulations_claude/gen.py` (generador), `pile` con `-R config_files`, ficheros de orden para garmonegro y CESGA, `submit.sh`, `run_local.sh`, `analyze_cluster2.py`.	• Un TOML = una ejecución = un sistema; los barridos son ON REQUEST. • Nada específico de Slurm, pero el manejo de señales y la reanudación lo hacen muy adecuado para colas con límite de tiempo.

→ **SQUAD** **D** *sin tocar* `nqs`: un generador que escriba N TOML (uno por punto y tamaño) y un `pile -R` que lance `nqs-run config.toml`. Es un buen primer proyecto en el fork, porque no choca con nada.

4.11 Postprocesado

Hydra	SQUAD
• **`aplot`**: índice de todos los JSON de resultados (en caché), consultas `--where` de pandas, `--app`, `--for`, `agg`, ajustes lineales (tamaño finito, extrapolación en varianza), trazas con suavizado y schedule superpuesto, tablas md/tex/csv. Guarda junto a cada figura un csv con el comando y la versión de git. • `reeval.py`, `refs.json`, `autoplot/energyVSirrep`, `plot_phase.py`, `plot_checkpoint.py`, `dynamic_plots`.	• Sin gráficas en el paquete (ni siquiera depende de matplotlib); hay un estilo `docs/squad.mplstyle` para las figuras de la documentación. • Lectura con checksums: `read_manifest`, `read_stage`, `read_inference`. `metrics.csv` y `observables.json` se abren con cualquier herramienta. • La arquitectura prevé clientes externos que lean los directorios de ejecución.

> → **SQUAD** **D**, el atajo con más rendimiento: añadir a `autoplots/index.py` un lector de directorios de SQUAD que genere una fila por (ejecución, stage) a partir de `manifest.json` + `result.json` (+ `observables.json` si hay inferencias), y la traza a partir de `metrics.csv`. Correspondencias: E → `final_energy_mean`; var → `final_energy_variance`; vscore → `final_v_score` (otra definición, §5); parámetros → `manifest.configuration.system.hamiltonian`; net → `…backbone.name`; irrep → `…head.sector.m`; epochs → Σ `n_steps`; seed → `run_seed`. Así puedes comparar en la misma figura resultados de Hydra y de SQUAD. Normaliza las unidades (E_{spin}) al indexar.

5. Diferencias de convención que cambian los números

Imprescindible antes de comparar un resultado de Hydra con uno de SQUAD.

Cantidad	Hydra	SQUAD	Consecuencia
Unidades de Heisenberg	Pauli por defecto ($\sigma\cdot\sigma$)	Operadores $S = \sigma/2$	$E_{\text{Hydra}} = 4\cdot E_{\text{SQUAD}}$
Signo del campo en TFIM	$+h\,\Sigma\sigma^x$	$-h\,\Sigma X$	Mismo espectro (unitaria Π Z); los estados difieren en el signo.
V-score	$N\cdot\text{Var}/E^2$	$N\cdot\text{Var}/(E-E_\infty)^2$	Coinciden sólo si $E_\infty = 0$. Con S^z fijo, $E_\infty \neq 0$ en Heisenberg.
Etiqueta de momento	`irrep (n_y, n_x)`, eje 1 = filas «y»	`m = (m_x, m_y)`, x = fila (k // L_y)	Misma numeración row-major: $n_y \leftrightarrow m_x$. Signo del carácter verificado para `b82edde`; conviene repetir la comprobación con el `Sequivariant` actual (coset).
Forma de la proyección	`phase_mode="angle"` por defecto	$\psi = z\cdot\psi_{\text{head}}$ (= `"log"`); en m = 0, la cabeza sin cambios	Las funciones de onda son distintas con los mismos pesos.
Congelado	SR completo + máscara	SR restringido al bloque activo	Trayectorias distintas si hay módulos congelados.
Energía publicada	`E_best` = mínimo de estimaciones ruidosas (sesgado hacia abajo) en el mejor paso	E(θ final) re-estimada con muestras nuevas	Para comparar, usar `reeval` (Hydra) o `nqs-infer`.
Cadenas	1024 × 1, sin descarte	16 × n/16, 16 barridos de descarte	Régimen de muestreo distinto.
Semillas	`jax.random.fold_in` por eon	`numpy.SeedSequence` por stage	No hay paridad bit a bit entre repositorios.
MLP y ComplexHead	$\alpha\cdot N$; cualquier C	Anchuras absolutas; $C \geq 3$	Inicializaciones distintas.
Transferencia	Voraz por forma	Por ruta y estricta	Pueden copiarse hojas distintas.

6. Balance en ambos sentidos

Lo que Hydra tiene y SQUAD no

- Transformers (ViT2D, CvT/CvT2/CvT3, PCvT) y otras 15+ redes; cabezas OutputHead y DeepOutputHead.
- Proyección con coset, `phase_mode` log/site, Z2, `SequivariantX`.
- Composición arbitraria (Sequential, Transversal, SplitTraining).
- Redes autorregresivas con muestreo directo.
- Modelos: SSM, largo alcance, J-K-Γ, Heisenberg e Ising con campos generales.
- Reglas Exchange por capas, J1-J2, inversión global, mezclas; estados iniciales.
- Adam y modificaciones de optax; opciones de SR (solver, NTK, momentum, `chunk_size_bwd`); lr por rama; sigmoide.
- Precisión mixta float32/float64; multi-GPU.
- Keeper multicriterio con *recover*; SanityMonitor; LeavesGradient; gráficas en vivo.
- ED con Z2, vectores guardados, `ED_inspection`.
- Rényi-2, estadísticas de módulo y fase, magnetización escalonada.
- Barridos, generador de campañas, `pile`, análisis de clúster.
- `aplot`, índice, `reeval`; diagrama de arquitectura.

Lo que SQUAD tiene y Hydra no

- Validación completa y `--dry-run`; chequeos de consistencia física.
- Transferencia estricta con informe y ΔE en cada frontera.
- SR restringido (congelado correcto).
- Reanudación exacta y manejo de señales; lock; `failure.json`.
- Manifest con trazabilidad completa, checksums, escritura atómica y esquemas versionados.
- Logs estructurados (`run.jsonl`); comprobaciones de NaN y de muestreado frente a exacto.
- Medida de τ_{int}; V-score con E_∞.
- `nqs-infer`: valores exactos, fidelidad, S(q), correlaciones.
- Comprobación de que H conmuta con las traslaciones; ED por base de órbitas.
- Honeycomb, kagome y cúbica; espín S; bosones; Bose–Hubbard.
- Embedding one-hot; regla de Marshall verificada.
- Muestreadores Hamiltoniano y parallel tempering; chequeo de ergodicidad.
- ≈1130 tests (oráculos, conformidad, arquitectura); CI; ADRs; releases; documentación con derivaciones.

Ideas que merecería la pena traer de SQUAD a Hydra mientras convivan: SR restringido, informe de transferencia con ΔE, `n_discard > 0`, V-score con E_∞ y la comprobación periódica de la energía exacta en 4x4.

7. Hoja de ruta propuesta para el fork

Principios

1. **Fork de `develop`**, no de `main`: en `develop` están los ADR, el SOP y los oráculos. Añade el original como `upstream` y haz rebase a menudo.
2. **Trabaja con sus reglas**: una rama o PR por ítem, con su test y su documentación, y un ADR breve cuando tomes una decisión. Así todo es devolvible; una divergencia larga en el fork hace imposible la convergencia que buscas.
3. **Habla primero con Alpesh** sobre v0.5.0 (simetría y composición, ADR 0072). Propónle: el contrato de malla gruesa, el wrapper de proyección sobre un subgrupo (coset) y la API de observadores. Su regla ON REQUEST exige «un resultado concreto que lo necesite y el criterio físico antes del port»: tu trabajo en J1-J2 (sectores, CvT3 8x8–10x10) es exactamente ese resultado concreto.
4. **Calibra antes de migrar experimentos**: ejecuta el mismo caso en ambos (p. ej. J1-J2 4x4 con CNN + ComplexHead, sin congelar) y comprueba la tabla de la §5 (unidades ×4, V-score, energía final frente a mejor). SQUAD ya tiene `benchmarks/tfim_4x4_hydra_comparison.py` como plantilla.

Fases

#	Qué	Tipo	Dónde	Por qué en este orden
1	Hamiltonianos: Heisenberg con campos, Ising general, SSM, largo alcance, J-K-Γ (con colores de enlace por dirección). Test del espectro contra una matriz independiente.	D/A	`nqs/systems/hamiltonians`	Aislado; te familiariza con el flujo de PR y CI de SQUAD.
2	Samplers: Exchange por capas, ExchangeJ1J2, inversión global, mezcla con probabilidades; estados iniciales.	D/A	`nqs/sampling.py`	Necesarios para reproducir tus recetas.
3	Schedule sigmoide, activaciones extra, OutputHead/DeepOutputHead como cabezas.	D	`training/schedules.py`, `ansatz/`	Pequeños y sin riesgo.
4	Herramientas externas: generador de TOML para barridos, `pile` con `nqs-run`, lector de SQUAD en `aplot`/índice, visor en vivo de `run.jsonl`.	D	Fuera de `nqs`	Te permite *experimentar ya* con SQUAD sin esperar a nada.
5	Contrato de malla gruesa + proyección sobre subgrupo (coset) → ViT2D, CvT3, PCvT (+ `DepthwiseConv`), con sus tests de equivariancia como tests de contrato.	A/R	`ansatz/`, `symmetry/`; ADR conjunto	Es el bloqueo principal para tu física; depende de v0.5.0.
6	Opciones de entrenamiento: opciones de SR (solver, `chunk_size_bwd`, momentum, NTK), optimizador, lr por rama. Precisión mixta (ADR + benchmark).	A/R	`vmc.py`, `training/periods.py`	Necesario para 8x8+ y para garmonegro.
7	API de observadores → keeper (instantáneas registradas), LeavesGradient, monitor por módulo.	R	`training/`; ADR	Diagnóstico avanzado una vez estable lo anterior.
8	Z2 y ED con Z2 (si v0.5.0 no lo trae); Rényi-2 y módulo/fase en `nqs-infer`; multi-GPU probado.	A	`symmetry/`, `reference/`, `inference/`	Completa la paridad con tus estudios de sectores.
9	ARViT y muestreo directo.	R	`sampling.py`, `ansatz/`	Baja prioridad.

Qué no portar

Los 4 JSON y el patrón `selection`; los códigos posicionales; `greedy_transplant`; el esquema de etiquetas/UUID (lo sustituyen `run_directory` y el generador); el envoltorio `Traslation`; CvTaps, CvTexp y VViT; los scripts de análisis del formato antiguo; el *recover-from-best* como comportamiento por defecto.

8. Anexo: correspondencia de configuración Hydra → TOML de SQUAD

Hydra	SQUAD	Comentario
`config0.write_folder_sim`	`run_directory`	Debe estar vacío.
`config0.seed`	`run_seed`	Derivación distinta.
`config0.sz_total`	`[system.hilbert] total_sz`	
`sampler.metropolis.sampler.rules` / `prules`	`[system.sampler] name`	Sin mezclas (fase 2).
`n_chains_per_rank`, `n_samples_per_chain`	`n_chains`; `[[stages]] n_samples`	Por stage.
`chunk_vstate`	`[[stages]] chunk_size`	
`VMC.vmc_sr_netket.*`, `optimizer`	–	Fijos (fase 6).
`network_dtype`, `matmul_precision`	–	float64 (fase 6).
`schedule.epochs_struct[e][r][p]`	`stages[e].periods[p].n_steps`	Las «eras» desaparecen.
`modes_struct`	`trainable_prefixes`	Rutas en vez de códigos.
`lr_struct`	`learning_rate` (número o tabla)	Uno por periodo.
`diag_shift`	`periods.diagonal_shift`	Partir las curvas globales.
`callback.keeper` / `checkpoint`	Checkpoints automáticos	No hay keeper.
`config1.sizes`, `*_list`, `mode`	Un TOML por tamaño y punto	Generador externo.
`kwargs_lattice.bc` / `order`	`[system.lattice] periodic` / fijo row-major	
`CM.selection` + parámetros	`[system.hamiltonian] name` + campos	Atención a las unidades.
`config2 stage.template`	`[stages.wavefunction.{embedding,sign_rule,backbone,head}]`	Composición limitada hasta v0.5.0.
`config3.storage[clase]`	Campos dentro de la tabla del componente	
`stage.load`	`transfer_rules`, `fresh_prefixes`, `discard_prefixes`	Estricto.
`load_model.load_from`	`initial_parameters = {from, stage}`	Sólo del final de un stage.
`change_attr`	Nueva especificación del stage	Explícito.
`symm_wrapper` (Z2), `Sequivariant.irrep`	– / `head.sector = {m=[m_x,m_y]}`	Z2 en v0.5.0.

Informe elaborado leyendo el código y la documentación de ambos repositorios (sin ejecutar simulaciones). Fuentes principales: `NNs/docs/01–11`, `NNs/Hydra/**`, `SQUAD/README.md`, `docs/{user_manual,architecture,developer_guide}.md`, ADR 0002, 0028, 0071 y 0072, y los docstrings de `nqs/**`.